

**Robot Sumo Simulation:
Design Document**

Team Members

Adebare Kofo-Abayomi: akofoaabyomi2022@my.fit.edu

Avlynn Wade: awade2024@my.fit.edu

Faysal Zaki: fzaki2023@my.fit.edu

Joshua Oziel Fernández Alvarado: jfernandezal2023@my.fit.edu

Client & Faculty Advisor

Department of Electrical Engineering and Computer Science

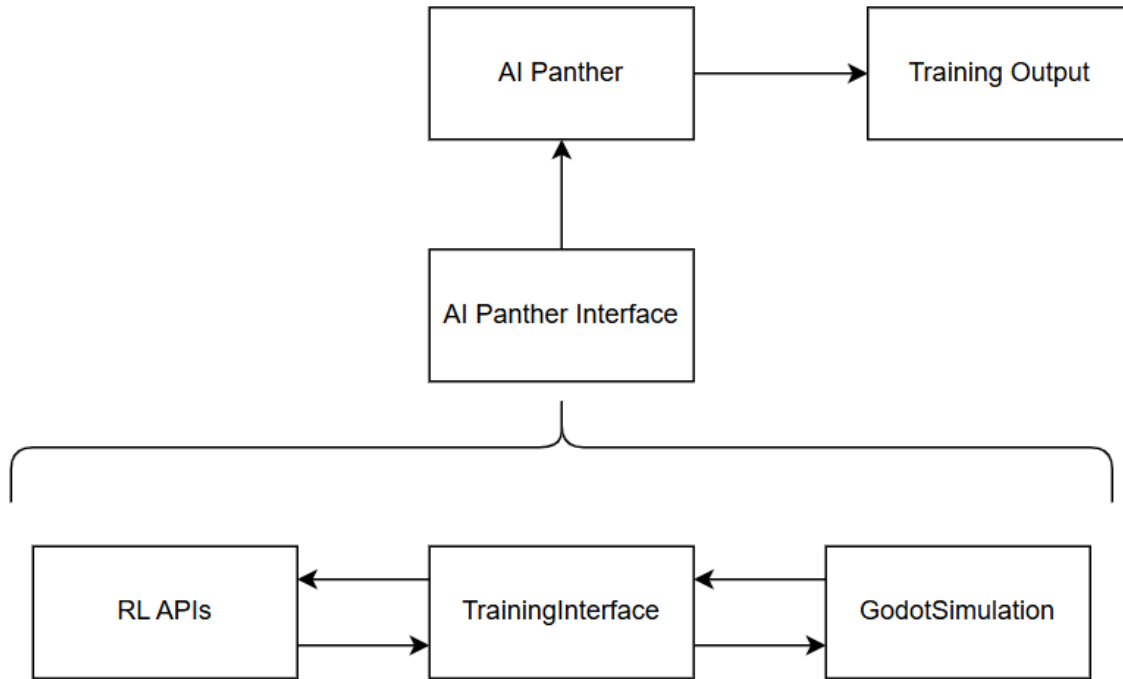
Dr. Hanks, thanks@fit.edu

Table of Contents

1. Introduction
 - 1.1 User Types
 - 1.1 User-Facing Machines
 - 1.1 Non-User-Facing Machines
2. Mock GUI
3. Algorithms
 - 3.1 Godot Simulation Algorithms
 - 3.2 API Algorithms
4. API Integration
5. Data Management

Section 1. System Architecture

The system overall is composed of multiple self contained machines which are connected through interfaces so that they may interact. The overall structure is composed of a Python script responsible for managing the training interactions between the RL APIs and the simulation. All tasks and computations during training will be delegated to the supercomputing resource (AI Panther in the developer's case), which will provide the output used as data for reproducibility and tracking.



Section 1.1. User Types

It is worth mentioning that the different user types can all overlap with each other, as developers also encapsulate all the other user types. Some users may be Trainers and Visualizers, while other users may only interact with the system as Players. Any user of the system can be described as a combination of the following user types.

I. Developers

During the initial phases of the project's life, the developers are limited to the project team (Adebare Kofo-Abayomi, Avlynn Wade, Faysal Zaki, and Joshua Oziel Fernández Alvarado) as well as the client and faculty advisor Dr. Tyler Hanks. Developers are expected to also interact with the system as all other user types.

Developers will have access to the system's GitHub repository and have the ability to work on the project's development in all of its aspects, from the Godot simulation to the RL

training. Developers will additionally have access to a number of debugging tools to ensure the system works as expected and to facilitate performance analysis.

Developers are expected to have competency with all of the system interfaces, outlined specifically in the Requirements Document [1] but broadly defined as the user of the Godot Engine, Python, the Gymnasium / Petting Zoo API, AI Panther and GitHub. Noticeably, Developers will be excluded when considering the difference between user facing vs. non-user facing machines, since developers will interact with all machines regardless.

II. Trainers

Trainers are the users that will execute and delegate training jobs utilizing the system's given interfaces. Trainers are expected to have a reasonable understanding of how RL training works and the algorithms utilized with the chosen APIs. They must be able to evaluate the efficiency and effectiveness of training, as well as correctly judge when training is being productive and when it is ideal to change the model or perform a new training run in order to avoid wasting computational resources.

Trainers are encouraged to perform training utilizing a supercomputing resource, which in the case of Florida Tech students and the development team will be AI Panther. However, if the Trainer wishes to utilize a different supercomputer resource they are capable of doing so as long as they correctly integrate the system's interfaces with their supercomputing resource of choice.

III. Visualizers

Visualizers are the users that will interact with Godot's simulation in order to visualize and run singular instances of a Robot Sumo match. Visualizers are expected to be able to use the Godot simulation along with all of its features. Visualizers will be able to import specific instances of training runs utilizing the exported output from the training interface in order to visualize them. They may additionally import specific agents into the simulation in order to visualize their performance in a new, seeded or unseeded, robot sumo round. Finally, Visualizers will have the ability to change the parameters of a round's specifics, including the robots, number of Players, and agents to participate.

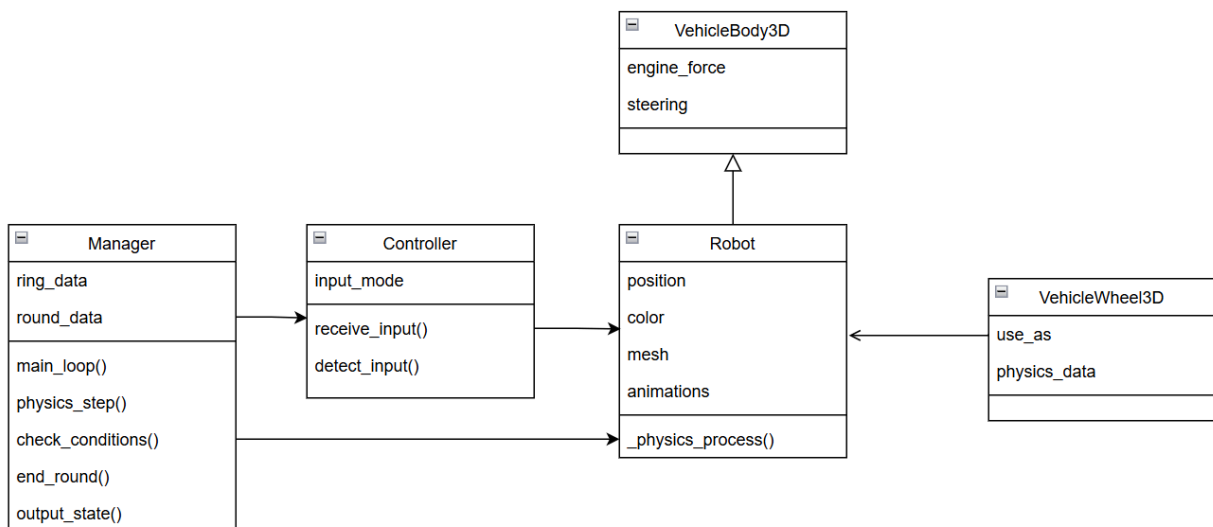
IV. Players

Players are the users that will participate in a robot sumo round within the Godot simulation against another Player or against an agent. The purpose of Players is to evaluate the performance of an already trained agent or to otherwise influence an agent's training by encouraging certain behaviors during a robot sumo round. Players are expected to understand the rules and concepts of a robot sumo wrestling round, as well as have an understanding of how the a controls, moves, and interacts with other robots. Players must be familiar with or otherwise learn to use the system's keyboard inputs in order to control the robot, which will follow a conventional video game control scheme.

Section 1.2. User-Facing Machines

I. Robot Simulation

The robot simulation will be developed inside Godot, utilizing its nodes and process architecture. Noticeably, Godot has built-in `VehicleWheel3D` and `VehicleBody3D` classes which are integrated into the physics engine, allowing the development team to use them as black boxes in order to define the physics behavior of the robots. All calculations, inputs and condition checks will take place inside each node's custom physics process in order to keep the system deterministic and consistent, avoiding frame vs physics offsets. This will be especially useful when designing the headless mode for training, since any functionality outside of the physics process will be lost.



II. Supercomputing Resource of Choice

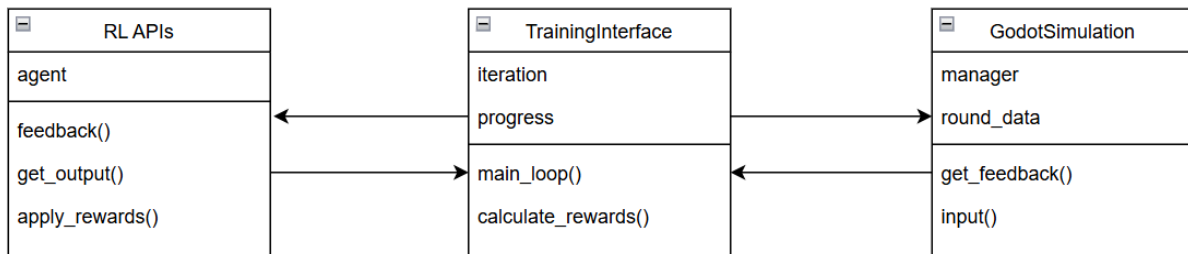
Users that perform training are encouraged to use a supercomputing resource to perform training because of the expensive computational costs associated with it. Although the user can use any resource they desire, the development team will be using AI Panther specifically. It is up to the user to correctly define the interface between the training API and the resource. The developers will define an interface specifically for the use of AI Panther. This will be defined in more detail as the project evolves once the developers have received access to AI Panther.

Section 1.3. Non-User-Facing Machines

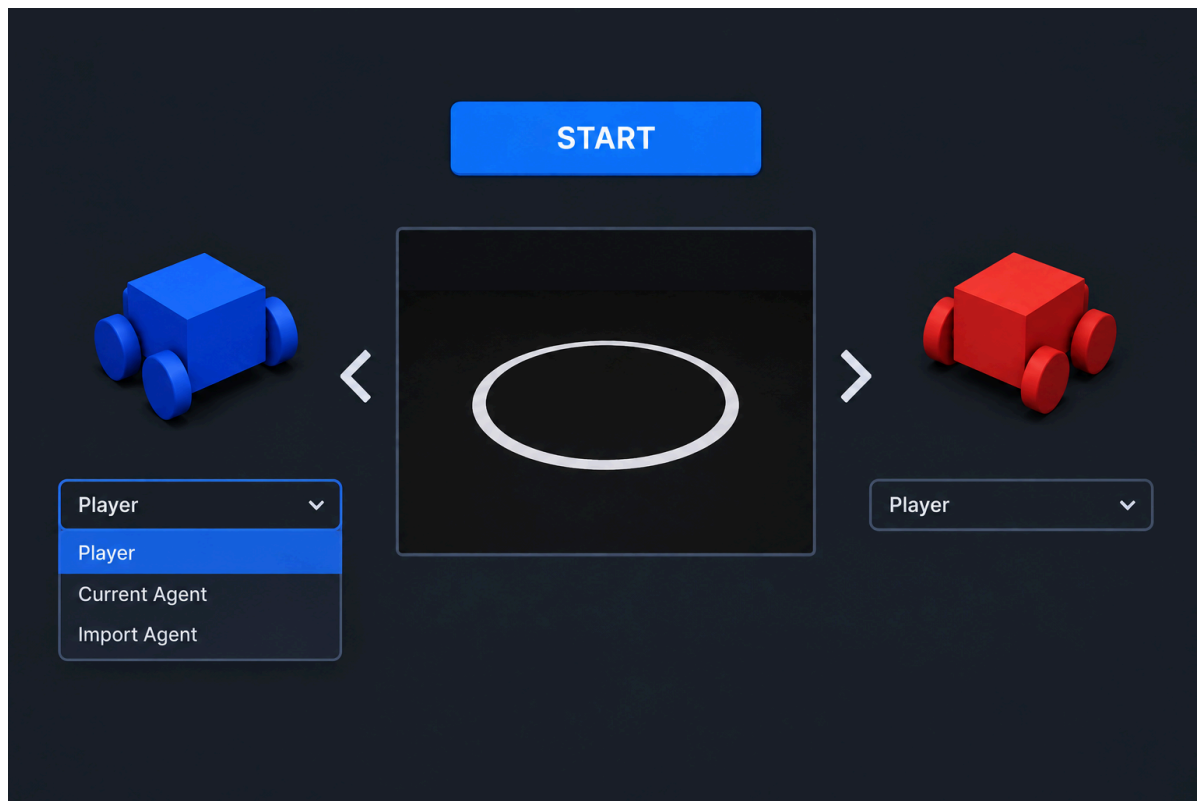
I. Gymnasium / Petting Zoo

The Gymnasium / Petting Zoo frameworks (RL APIs) will be used to determine the behavior of the trained agent's learning. These will be connected to the simulation through an

interface defined utilizing Python, allowing the RL APIs to receive from and provide information to the Godot simulation. Information will be received as full state feedback initially, but may optionally be expanded to sensors later as scope allows. The agent will make a decision and return it as a value that can be interpreted by the simulation's Manager as an input for the Controller; the Manager will execute its loop once and return the new feedback to the RL APIs, which will continue the training loop. Positive and negative rewards will be given to the agent during and after every round of simulation, which will be interpreted by the RL APIs to adjust the agent.



Section 2. Mock GUI



Section 3. Algorithms

Section 3.1 Godot Simulation Algorithms

I. Physics

The simulation will use the default Godot physics engine for 3D projects, Jolt Physics. The physics engine is integrated with all provided classes and methods regarding colliders and bodies, so there is no reason to rewrite the physics. If necessary, specific physics behavior can be overwritten by directly modifying the engine code and creating a custom version of Godot, since it is an open source software.

The main objects that will require physics are the Robots. Robots will be defined as a class inheriting from `VehicleBody3D` with `VehicleWheel3D` children. The wheels will automatically affect the direction of movement and behavior with collisions according to their suspension, radius, grip, and other values. The wheels contain a variety of physics values that must be tuned through experimentation and analysis of real sumo robots. Game world objects like the floor will be defined as monitorable physics bodies that the robots can interact with.

II. Robot Controllers

Robots will be controlled using a custom Controller class responsible for translating directional input into engine force and steering that can be interpreted by the `VehicleBody3D` class in order to execute movement and interact with the physics. The Controller classes will be able to receive input as both a directional vector or through keyboard inputs. In the round setup screen, the user will be able to select an option for each robot and the Controller will behave accordingly. Each Robot will have their own Controller node, and it is the responsibility of the Manager to correctly distribute and interpret the inputs received from the agent's API output when a non-player Controller option is chosen.

III. Main Loop

The default process and physics process modes for the project will be set to disabled. If the user starts a round to visualize it, the Manager's process and physics process will be enabled exclusively so that it can run its main loop. The Manager is then responsible for calling the respective methods on the rest of the nodes in order to keep the simulation deterministic.

When the game is run without visualization through the training interface, the Manager will not run through either of the engine's processes. Instead, the training interface will call a step method on the Manager which acts as a physics process call. The Manager can then return the state feedback to the training interface. This will enable the supercomputing resource to repeatedly call the physics steps while fully disabling visualization in order to focus exclusively on the physics calculations and save on computational resources.

Section 3.2 API Algorithms

The specifics of how the RL APIs interpret rewards and build a neural network will be treated as a black box, since it is out of the project's current scope to define how exactly RL training occurs. However, the development team will explore and evaluate options and interfaces with which the TrainingInterface can interact in order to provide feedback and receive inputs.

Trainer users may modify the TrainingInterface in order to use their own RL API to conduct training with or when using the Import Agent functionality in order to bring an already trained agent.

Section 4. API Integration

To perform training, the TrainingInterface class will act as a connection between the RL API and the Godot Simulation. The TrainingInterface class will be responsible for stepping through the simulation's looping and stepping functionality, gathering the feedback from the simulation, manipulating the data in order to provide it as input for the model, and receiving the output of the agent's decision. It will also modify the agent's output in whatever capacity necessary so it can be passed into the simulation conforming with the input guidelines defined in the Manager and Controller classes.

When performing training, the TrainingInterface must also evaluate the feedback from the simulation and calculate a set of positive and negative rewards which can then be given to the model in order to update its weights. The rewards will be defined in more detail as the simulation starts to take shape, but the general trend will give positive rewards to good performance within the rules of Robot Sumo, and negative rewards to actions that actively hinder the robot's own position. As an example, proximity to the center of the ring can be a positive reward, as well as pushing an opponent closer to the edge of the ring. Similarly, moving towards the edge when an opponent is not there can be interpreted as a negative reward. Rewards will be evaluated and tweaked during training as hyperparameters in order to improve the agents.

Section 5. Data Management

One of the most important objectives of this project is to ensure deterministic results in order to facilitate debugging, training analysis and reproducibility. The primary method to be used for this will be seeding and output data. Every training run and individual godot simulation will utilize seeds. By doing this, any individual simulation or training run can be replicated by simply reusing the seed.

All training runs will be logged in a data management system like JSON files. As training is executed using AI Panther, simulations will output their ending state and be compiled as part of a larger file containing all simulations that took place during training. This will contain a list of seeds, round results, and other data deemed necessary like information about the agent's current training or the decisions the agent is taking. What data is part of the output can be modified per training session as needed, since developers may need to take a look at different things depending on what issues are being encountered.

By reading the data outputted during training, a specific round that was played non-visually may be reproduced exactly by the simulation utilizing its seed and other encapsulating information like the agent's inputs. This will be useful for analysis and pinpointing certain behaviors that a Trainer may want to encourage or discourage.

When using AI Panther, data must be downloaded and stored by the developers in order to ensure it is not lost by a purge. All data regarding training and retired agents will be stored in a cloud so it can be used as reference when making decisions about what to change regarding an API or how rewards are calculated.